

The Mathematics of Vector Search in RAG

Karina Leśkiewicz

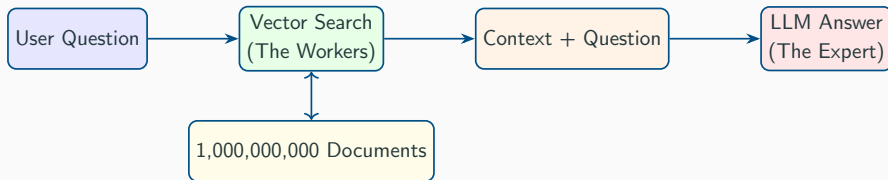
May 24, 2026

1. Introduction

What is RAG? (The Factory Analogy)

Think of RAG like a specialized factory. We have billions of files and a genius expert (the LLM).

- **The Warehouse Workers (IVF/HNSW):** Their only job is to find the right file in milliseconds. They don't need to "understand" the data; they just follow coordinates.
- **The Expert (LLM):** Once the worker brings the file, the expert reads it and answers the user.



How do we measure "Meaning"?

In AI, text is turned into a list of numbers called a **Vector**. If two sentences mean the same thing, their vectors are physically close together in space.

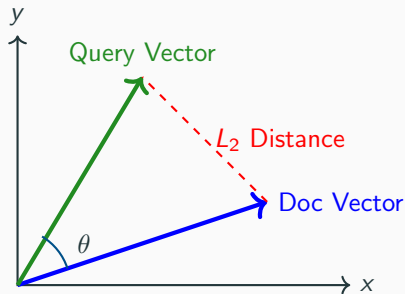
Euclidean Distance (L_2):

The straight-line gap between two points.

$$d(q, x) = \sqrt{\sum_{i=1}^D (q_i - x_i)^2}$$

Cosine Similarity (θ):

Measures the angle between vectors. (Most LLMs prefer this!)



The Curse of Dimensionality

In 2D space, points can be close or far, but in high-dimensional spaces (e.g., 768D), everything is practically the same distance apart, making exact search useless.

By the law of large numbers, the pairwise distances between random vectors in high-dimensional space concentrate around the mean:

$$\lim_{D \rightarrow \infty} \frac{\text{dist}_{\max} - \text{dist}_{\min}}{\text{dist}_{\min}} = 0$$

Conclusion: Exact k -NN ($O(N \cdot D)$ complexity) fails at scale. We must use **ANN** (**Approximate Nearest Neighbors**) to trade a small loss in Recall for a logarithmic speedup.

2. The Search Shortcuts

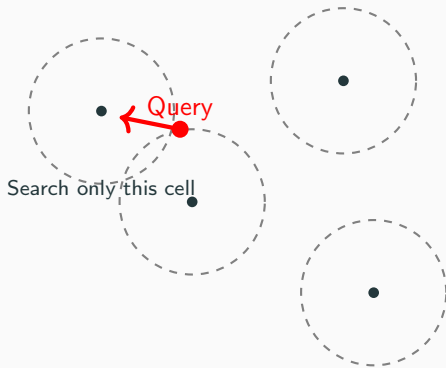
Shortcut 1: IVF (Clustering the World)

The Idea: Don't look at every document. Group documents into "cities" (clusters). When a question comes, find the closest city and only search inside it.

How it works:

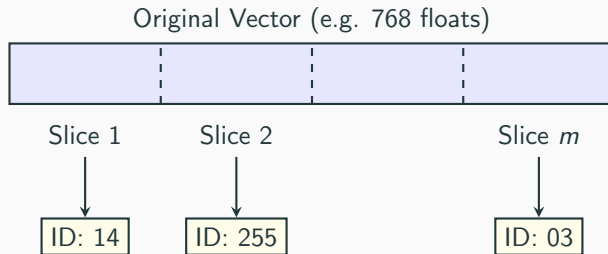
1. Use **K-Means** math to find center points (Centroids).
2. Every document is assigned to its nearest Centroid.
3. This creates Voronoi Cells (mathematical boundaries).

Benefit: Instead of searching N documents, we search only a small fraction (e.g., 1%).



Shortcut 2: PQ (Slicing & Dicing)

The Idea: High-dimensional vectors are huge (768 numbers). To save RAM, we chop them into pieces and replace each piece with a short "ID code."



The Trade-off: We minimize the error between the original and the ID centroid:

$$\mathcal{E} = \sum_{j=1}^m \|\text{Slice}_j - \text{ID_Centroid}_j\|^2$$

Shortcut 3: ADC (Fast Math Cheat Sheet)

The Idea: If our database is compressed into IDs, how do we calculate distance? We build a **Cheat Sheet (Lookup Table)** for the query.

The ADC Process:

1. Pre-calculate the distance from the *Query* to every ID in the dictionary.
2. Put these in a table.
3. For any document, just **look up** the IDs and sum the answers!

Distance Lookup Table (Cheat Sheet)

ID	Dist to Chunk 1	Dist to Chunk 2
ID: 01	0.45	1.20
ID: 14	0.12	0.66
ID: 255	0.99	0.05

If DB Doc has IDs [14, 255], total dist = 0.17

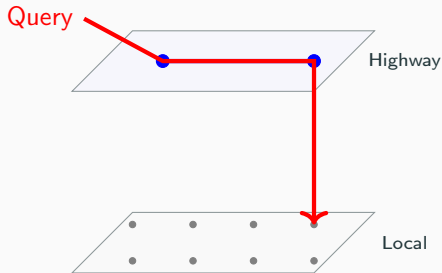
Shortcut 4: HNSW (The Highway Graph)

The Idea: Searching a graph is like finding a house. You don't drive every street; you take the highway to the right city, then use local roads.

Graph Layers:

- **Highway:** Very few nodes, huge jumps.
- **Local:** All nodes, tiny jumps.

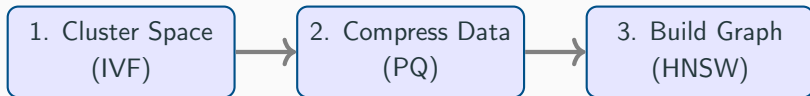
Process: Hop across the highway to the node closest to your query, then drop down a layer to get more detail.



3. Summary

The Complete Process: How it all Connects

Phase 1: Preparing (Offline)



Phase 2: Searching (Online)

